



IIC2173 2025-2

Examen

Nombre:

Número de alumno:

Fecha:

Instrucciones

- Tiene **2:15 horas precisas** para completar este examen.
- Tendrán **15 min** para escanear las hojas de este examen y subirlas a Canvas en la tarea *EX*. **NO** habrá excepciones a este plazo de escaneo y subida. Posteriormente, entregue sus hojas al profesor/ayudante para su archivado. No escriba más en este plazo, ni comente con sus compañeros.
- Recuerde nombrar todas sus hojas.
- Solo se responderán preguntas realizadas en la sala, en los momentos destinados para ello.
- La prueba es estrictamente individual. No copie. La idea es tratar de evaluar los conceptos y que aprenda los principales contenidos del curso.
- Este curso se adscribe a la política de integridad académica del DCC, que se encuentra en el programa del curso. Esperamos que respeten la política de integridad académica, y los exámenes serán comparados con diversos métodos para revisar casos de copia.
- Puede usar las hojas que necesite para completar la interrogación, pero deben ser precisos en sus respuestas para facilitar el entender qué quieren decir. Piense que el ayudante y profesor deben

" Lo que te impide actuar, impulsa la acción. Lo que se interpone en el camino, se convierte en el camino"

(Meditaciones, Marco Aurelio Antonino, emperador Romano)

1) Preguntas de Desarrollo (20p)

Escriba sus respuestas en el espacio, máximo 8 líneas en preguntas a y b. De escribir más, solo se corregirán hasta las líneas permitidas.

1.a) (8p) Comente detalladamente sobre la siguiente proposición:

“El patron cliente-servidor es el estilo microkernel con binding runtime”

Respuesta: El alumno debe interpretar que esta proposición es falsa, debido a que es una sobre simplificación y tal como se puso demanda equivalencia precisa. Es posible que se pierda en que es una analogía, pero solo es eso: analogía. Debe mencionar que...

- En microkernel el estado interno del core puede ser traspasado a los plugins, y eso no pasa en cli-sv
- Microkernel demanda que los plugins introduzcan funcionalidad al core, cosa que no necesariamente pasa en cli-sv
- En cli-sv la lógica de negocio es implementada por el servidor, en microkernel el plugin es quien la implementa principalmente

Habiendo tantos caveats, no se puede hacer la afirmación -> incorrecto. Debe mencionar 2 de 3

1.b) (6p) Muchas de las tácticas para promocionar integrabilidad se parecen a las tácticas para promocionar la modificabilidad (e.g. encapsular, middleware, etc). ¿Al diseñar un sistema altamente modificable, automáticamente implica que será fácil de integrar en otro contexto? Justifique

Respuesta: Esta afirmación es en estricto rigor falsa. Un sistema altamente modificable no implica que orientó sus facilidades a la integrabilidad. Puede ser modificable internamente pero no tiene capacidades de comunicación, o no se ofrecen interfaces claras al exterior, usar protocolos propietarios, descontrol en versionado, etc. Puede que se facilite, pero no es un automático por solo orientar el sistema a una característica arquitectónica.

1.c) (6p) Una empresa de E-Commerce está preparando el despliegue de una nueva versión de su servicio principal. Este servicio recibe altos volúmenes de tráfico, funciona en una arquitectura de microservicios y debe mantenerse disponible 24/7. La empresa teme que la nueva versión pueda contener errores no detectados que afecten la disponibilidad o la experiencia de usuario.

Explique cómo diseñaría una estrategia de despliegue que permita introducir la nueva versión minimizando el riesgo y manteniendo la continuidad operacional del sistema. En su respuesta:

1. Compare el enfoque de Canary Release con Blue/Green Deployment, indicando ventajas, desventajas y en qué casos cada uno sería más adecuado.

2. Diseñe un plan concreto de *deployment* para este servicio utilizando uno de los dos enfoques, justificando por qué es el más apropiado para el escenario y definiendo que métricas usaría para que sea segura.

Respuesta: *Canary Release implica el uso de ambos ambientes (legacy y nuevo) y que se distribuyan los usuarios para que usen uno u el otro, permitiendo probar el nuevo ambiente con cargas graduales de usuarios a cambio de tener dos versiones en uso. Blue/Green Deployment implica actualizar todo un ambiente para luego dirigir todas las requests al nuevo ambiente, permitiendo hacer un cambio directo para los usuarios a cambio que si hay un problema lo vean todos.*

Se espera que se indique que es mejor el Canary Release por poder probarlo con pocos usuarios y haciendo cambios graduales. Se espera que la métrica indicada pueda ver bugs en la plataforma, ya sea de uso de usuarios o errores críticos.

- *2 ptos por comparar canary release mediante sus ventajas y desventajas (no es necesario describirlos con mucho detalle)*
- *2 ptos por describir blue green deployment con sus ventajas y desventajas*
- *2 ptos por indicar correctamente el enfoque y su justificación*

2) Preguntas de Alternativas (6p)

Escoja la alternativa correcta para cada pregunta.

2.a) (2p) En el diseño de un sistema para gestionar reservas de salas, ¿cuál de las siguientes alternativas representa correctamente el conocimiento de dominio y no conocimiento técnico?

- I. Reglas como “una sala no puede reservarse dos veces al mismo tiempo”
- II. Lenguajes de programación y frameworks que conozco.
- III. Qué departamentos pueden reservar salas VIP.

- a) Solo II
- b) II y III
- c) I y III**
- d) Todas las anteriores

2.b) (2p) ¿Cuál de las siguientes alternativas describe correctamente una diferencia entre REST, GraphQL y gRPC?

- a) GraphQL solo permite operaciones de lectura, a diferencia de REST y gRPC que permiten lectura y escritura.

b) gRPC permite comunicaciones bidireccionales, mientras que REST y GraphQL son unidireccionales

c) gRPC es exclusivamente para comunicación web entre navegadores, mientras que REST y GraphQL son solo para comunicación entre backend y backend.

d) REST requiere que el cliente defina exactamente qué datos quiere recibir, mientras que GraphQL siempre devuelve todos los campos del recurso por defecto.

2.c) (2p) ¿Cuál de las siguientes afirmaciones describe correctamente una diferencia clave entre cookies de sesión y OAuth?

I. Las cookies de sesión se utilizan para mantener la sesión de un usuario dentro de una misma aplicación, mientras que OAuth es un protocolo que permite delegar acceso entre aplicaciones o servicios sin compartir credenciales.

II. Las cookies de sesión permiten delegar permisos entre distintos sistemas, mientras que OAuth solo funciona dentro de un único dominio.

III. OAuth solo funciona en navegadores web, mientras que las cookies de sesión están diseñadas exclusivamente para aplicaciones móviles.

a) Solo I

b) Solo II

c) II y III

d) Todas son correctas

3) Pregunta de Caso (34p)

Escoja uno de los dos casos disponibles, y responda sus preguntas *a, b y c*. Luego, responda las preguntas comunes *d, e, f y g* con el caso que escogió. Hacer dos preguntas **no** tiene bonus disponible y solo se corregirá la que indique. **Si no indica alguna, no se considerará respondida**.

Caso I

MacroHard es una casa de software muy conocida que genera sistemas operativos para cientos de millones de instancias de computación, sean PCs, servidores corporativos, entornos embebidos, etc. Toda esta variabilidad en entornos de computación ha generado una fuerte necesidad de testear los *releases* de sistemas operativos en cada una de estas plataformas.

LegitBusiness le ha encargado la creación de una plataforma de *testing* capaz de coordinar estos esfuerzos (de *testing*) en un universo de 3500 variantes de plataformas. Cuando los equipos de desarrollo generan una versión *nightly*, que acumula varias *features* que deben ser probadas, se debe generar un *build* adaptado a cada una de estas plataformas junto con un ecosistema de *mocks* de diversos sistemas de MarcoHard que deben funcionar. Cada plataforma debe ser capaz de obtener el *build* requerido con el que es compatible, instalar ese *build* y ejecutar las diversas baterías de *tests* especificadas a continuación de forma secuencial:

- Test incorporados: Ciertos *system checks*, *sanity checks* y *tests* de integración sobre programas son ejecutados en cada máquina. El estado y resultado de estos *tests* debe ser reportado para su observación.
- Pruebas de interacción automáticas: Cada dispositivo tiene un adaptador simulador de “*Human interface devices*” (HID) incorporado el cual es capaz de ejecutar una serie de instrucciones (e.g. tipear, hacer click en cierto punto, scrolling, tomar screenshots, etc). Este dispositivo HID recibe la señal de que debe comenzar a testear desde la plataforma y ejecuta los *tests* que se hayan especificado desde la máquina.
- Pruebas de BSOD (o kernel panic): Las plataformas son probadas en escenarios críticos de falla intencional. Cada máquina reportará como se recuperó después de reiniciarse de forma automática. Si no puede reiniciarse en un tiempo apropiado, el dispositivo HID reiniciará el computador y si el problema persiste, se marca la máquina como defectuosa y se registra el problema.

Todos los resultados de estos *tests* deben ser reportados a los desarrolladores en un sistema que puedan consultar. Modele un sistema que cumpla con estas características.

3.1.a) (4 ptos) Comente sobre las necesidades de escalabilidad y elasticidad de este sistema y si poseen alguna relación obligatoria entre ellas para este sistema.

Respuesta: *Este sistema tiene necesidades fuertes de elasticidad al momento de deployear cada build, en el sistema de distribucion de builds. Sobre la escalabilidad, no se requiere como atributo principal: el set de plataformas de fijo y estable, por lo que la demanda*

3.1.b) (4 ptos) ¿Qué tipo de fallas locales podrían escalar a fallas sistémicas en esta plataforma y cómo podría diseñar medidas de contención?

Respuesta: *Hay varias tácticas que pueden servir para contener fallas locales:*

- *Timeouts*
- *Redundancia en el agente de plataforma*
- *Graceful degradation: un bajo acoplamiento permite que el software evite perder funcionalidad crítica (como reportar estado a main) e informe de la degradacion de sus partes (que se desarme de a poquito)*
- *Sanity checks antes de ejecutar tests*
- *Rollback de la ejecucion de los tests*
- *Reparar estado: con agentes nativos (tipo restaurar sistema, sfc, chkdsk, etc) o alguna solucion custom*

El alumno debe proponer al menos dos y debe identificar que estas tácticas pueden ser escaladas a nivel macro, con cierto costo de mantenibilidad y complejidad

3.1.c) (4 ptos) Si quisiéramos incorporar nuevas plataformas en este sistema, que deberíamos considerar en el diseño de este sistema? No es necesario que implemente lo de esta pregunta en su diagrama de componentes

Respuesta: *El alumno debe fomentar la integrabilidad (y mencionarlo). Debe implementar al menos una o dos tácticas de integrabilidad, como usar algún estándar de comunicación con el kernel de sistema (como POSIX o similar). Podría diseñar el agente de plataforma como microkernel, donde tenga varios módulos que pueden interactuar con diversos tipos de kernels. Otras solucion puede ser generar un modelo abstracto para los tests que tenga un adapter detrás, cosa que el dev escriba un DSL como los que existen en ruby y que haga la traduccion del lenguaje a las diversas plataformas.*

Los 4 ptos requieren a lo menos dos sistemas

Caso II: Collector AI (nombre de marketing, aunque no use AI)

Existen muchas empresas hoy en día donde una de sus mayores preocupaciones son la cobranza hacia sus clientes, ya sea por morosidad o solo porque le enviaron la factura a fin de mes. Con esto en mente LegitBusiness, viendo su gran progreso con su aplicación de visitas a distintas propiedades, decidió encargarles el diseño de este proyecto dado que sus requerimientos ya fueron levantados por otros Ingenieros previamente. Así, esta aplicación ayudará a las empresas a poder generar las distintas facturas en base a los datos mensuales de sus clientes, permitirá revisar que estos también pagaron y deberá de consolidar que está todo correcto para que la empresa pueda ver su reporte de conciliaciones.

De esta forma, les informan que la aplicación, para mantener una correcta confiabilidad de datos, debe separarse en múltiples servicios, cada uno con su propio flujo interno y sus funcionalidades:

1. **Servicio de contabilidad:** Este servicio será la fuente de verdad de todo el proceso. Una vez que cada sistema termine su flujo le deberá notificar a este otro sistema la actualización de la información para que lo guarde, y luego este lo propague al servicio de acceso de datos. También deberá hacer los *matches* entre los pagos y las facturas para ver quienes han pagado y quienes no, y así poder entregar un reporte agrupado para cada empresa.
2. **Servicio de Acceso a datos:** Este servicio es el que todos los usuarios accederán para poder ver la información de las consolidaciones de su empresa en particular. Se pensó en separarlo para no sobrecargar el servicio de contabilidad. Debe de disponibilidad una API para recibir consultas de los usuarios (vía su *front* o directamente a su API) con un caché para evitar hacer búsquedas de consultas repetidas. Finalmente, debe permitir que los usuarios de empresas hagan una *request* por acá para solicitar una factura y que esta se propague al servicio de facturación para que comience ese flujo.
3. **Servicio de Scrapping:** Este servicio se encargará de conectarse a múltiples bancos mediante el uso de *Selenium* para que, simulando ser un navegador de un usuario normal, le entreguen un set de instrucciones y pueda ingresar las credenciales secretas de las empresas y así ver su cartola de transferencias, descargarlas y enviarlas al servicio de contabilidad para que se haga el match con las facturas.
4. **Servicio de facturación:** Este servicio esperará la solicitud del usuario desde el Servicio de Acceso a datos para empezar su flujo. Recibirá los datos a quienes cobrar, un correo y el monto para enviar la factura correspondiente, y luego de generarlo lo notificará al servicio de contabilidad y le enviará el cobro al correo indicado.

Todos estos servicios deberán modelarse correctamente para seguir un correcto funcionamiento del sistema, y como se indicó, su sistema debe presentar un *Frontend* para su Servicio de Acceso a Datos. Además, como pueden usarse varias empresas alrededor del mundo, se espera que este *Frontend* implemente un CDN para su replicación y mejora de Performance.

3.2.a) (4 ptos) El sistema puede tener errores en algún match o en sus mismos flujos, por ejemplo, el servicio de contabilidad puede presentar el problema de que encuentre un pago sin asociar a una factura. ¿Qué medida tomaría para poder tener un control de esto, que se levanten todos los posibles errores, y que finalmente una persona encargada sea capaz de revisarlos para corregirlos? Indique la medida, cómo la implementaría en este sistema y justifique su respuesta.

Respuesta: Se espera que indiquen algún tipo de monitoreo donde se recopilen los errores y permita que una persona los vea después. Ya sea un servicio externo o servicio propio, se debería de indicar la forma en que todos los servicios se puedan comunicar con este para recopilar la información.

También puede ser válida otra medida que pueda generar esto.

- 1 pto por indicar la medida
- 1 pto por indicar la implementación correctamente
- 2 ptos por justificar correctamente

3.2.b) (4 ptos) El sistema, como contempla tantos servicios distintos, necesita que se comuniquen fácilmente para no tener que modificar esas integraciones continuamente y evitar altos niveles de acoplamiento. Indique, en base a las medidas de integración que se vieron en clases, cuál sería la mejor opción para esto y por qué.

Respuesta: Dentro de las clases de integrabilidad se encontraron Middlewares, MOM, REST, gRPC y GraphQL, dentro de los cuales MOM sería la mejor opción. Se espera que la justificación indique el cómo se vería el sistema sin esta medida y que, al ser un broker de mensajería puede modificar los envíos de datos para que funcionen a través de este sistema, y así que no se genere una conexión de todos con todos, si no que a través de esta entidad.

- 2 ptos por indicar el uso de MOM
- 2 ptos por justificarlo correctamente

3.2.c) (4 ptos) Como les falta un sistema de autenticación, les indican por un lado que deberían usar el protocolo OAuth para autenticar a sus usuarios, mientras que otros les dicen que sí, pero lo mejor sería usarlo junto a JWT. Indique si estas dos opciones se pueden usar en paralelo o si es innecesario que se usen a la vez, justifique su respuesta, y luego indique cómo y en que parte implementaría el protocolo en este sistema.

Respuesta: Si se pueden usar ambos juntos, ya que JWT sería el formato del token entregado por el protocolo OAuth. Este sistema debería de implementarse a para el servicio de acceso a datos principalmente, ya que ese es el punto de acceso para los usuarios.

- 2 ptos por indicar que si se pueden usar juntos y justificarlo correctamente (no se dará puntos solo por decir que “si se puede”)
- 2 ptos por indicar correctamente su implementación en el sistema

Preguntas comunes a los casos

3.d) (3p) Indique las características arquitectónicas (al menos 3) que Ud. considere son fundamentales en este problema. Enumérelas en orden de prioridad y justifique brevemente.

Respuesta Caso I:

- *Válidos: Mantenibilidad, seguridad, confiabilidad, otro bien justificado.*
- *No aplican: Escalabilidad ni elasticidad (porque se definen 3.1.a), usabilidad, soporte, flexibilidad.*

Respuesta Caso II:

- *Válidos: Performance, escalabilidad, integrabilidad, mantenibilidad, seguridad, confiabilidad, soporte, otro bien justificado.*
- *No aplican: Portabilidad, flexibilidad, extensibilidad, usabilidad.*

3.e) (4p) Indique los estilos/patrones arquitectónicos (al menos 2) que va a emplear para solucionar el problema. Indique explícitamente qué aspecto del problema ataca cada estilo (responsabilidad).

Respuesta Caso I:

- *Válidos: Peer to Peer, Pipe & Filter, EDA, otro bien justificado*
- *No aplican: Main y subrutinas, Código móvil, Intérprete.*

Respuesta Caso II:

- *Válidos: SOA, EDA, Pub/Sub, En capas, En lotes secuenciales (solo para el flujo de scrapper), Basado en reglas*
- *No aplican: Microservicios (no son microservicios, al tener más de una funcionalidad cada servicio), Código móvil, Intérprete, Pizarra, Peer to Peer*

3.f) (13p) Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Diseñelo en una hoja aparte.

Respuesta: *Si no es un diagrama de componentes UML no se corrige. Como puede haber muchas soluciones al problema, se analizará en base a descuentos:*

- *(-2 c/u) No usan los estilos ni los componentes que mencionaron en preguntas anteriores*
- *(-2 c/u) Falta un requisito funcional*
- *(-1 c/u) Interfaces mal implementadas, max. 8 puntos de dscto. Esto es que estén al revés o que los conectores no tengan interfaces.*

- (-1 c/u) Componentes desagregados (sin conectores)
- (-2 en total) No hay nombres en interfaces importantes

3.g) (2p) Identifique 2 falencias de la solución de su diagrama y comente como las solucionaría y las implicancias de estas soluciones. No se corregirán aquellas que apunten al formato de cómo se diagramó o si faltó explotar algún componente. Escriba esta respuesta junto a su diagrama si le falta espacio.

Respuesta: Se esperan trade-off asociados a su solución, puede ser por una decisión de implementar un estilo u herramienta o por el mismo diseño de su sistema. Sobre la distribución de puntaje:

- (0.5 ptos) Identificar un trade-off correctamente
- (0.5 ptos) Identificar correctamente una posible solución aplicable a su sistema